

1 TypeSpecs Syntax Specification

Annotations	A	::=	$\overline{\text{@spec } E}$
Expressions	E	::=	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	::=	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll_ :n \gg \mid \ll_ :_ *n \gg \mid \ll_ :n, _ :_ *n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	::=	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names.)

Figure 1: TypeSpec

It is noteworthy that the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence. Furthermore, not only is $\text{@spec } \overline{E}$ non-empty sequential elements but also multi-line elements unlike the other comma-separated sequences. That is TypeSpecs only allows one specification on one line.

This specification addresses the most fundamental and atomic types based on the TypeSpecs reference on Elixir documentation. The addressed are, starting from the top type **any()** and the bottom type **none()**, controversially vague concept of field types of record types. While thus enables expressing every existing TypeSpecs type, some might come up with questions on fundamentality with subtype relations or notion of expressions between basic types and literal types. In the following sections, these questions are addressed for clarity with listing the built-in types and the syntactic sugars in Table 1 and Table 2.

1.1 TypeSpecs Basic Types and Literals

TypeSpecs is rather expressive especially by providing the literal types that are also used in the code with the same form. However, this is classic that the higher expressiveness brings the higher complexity and TypeSpecs is not exempt. The common trade-off must be discussed here due to the fact that our ultimate goal is to translate TypeSpecs to Elixir Types. In the perspective of translation, while TypeSpecs allows literals types such as $:k$, $[]$, $\ll$, $(\bar{T} \rightarrow T)$ and $n..n'$, Elixir Types will not support $\ll$, and the range type (i.e., $n..n'$) will not appear to users but will be only used internally. This difference will not allow direct translations for all the types and eventually necessitate approximations of them in Elixir Types.

The first question could arise is from the subtype relationship between **atom()** and $:k$. It is true that $:k$ is a subtype of **atom()** since $:k$ represents every single atom type and **atom()** represents a set of all atom types. Nonetheless, we have to contain $:k$ in the syntax specification. That is because $:k$ is a literal type that could be any single atom type, e.g., **true** and **false** are literals that are used as themselves in the code and also in type specification. Therefore, it is always sound for **atom()** to replace **true** or **false** but not vice versa.

It could be viewed that the range literal (i.e., $n..n'$) expresses **neg_integer()**, **non_neg_integer()** and **pos_integer()**. This idea could be backed by Types and Their Syntax in Erlang documentation that these are built-in types which can be thought defined by $..-1$, $0..$ and $1..$, respectively. But Elixir documentation specifies them as basic types and the provided range syntax in Erlang documentation is not correct use. Provided, it is sufficient to say that these are not subtypes of $n..n'$, hence they are necessary to be in the syntax specification.

1.2 TypeSpecs Built-in Types and Syntactic Sugar

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpecs Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpecs Syntactic Sugars

2 Elixir Types Syntax Specification

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infy..n n..infy { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{..., $\overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() bitstring() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{..., $\overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Types Syntax

2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
<code>boolean()</code>	<code>:true</code> or <code>:false</code>
<code>number()</code>	<code>integer()</code> or <code>float()</code>
<code>non_empty_list(T)</code>	<code>non_empty_list(T, empty_list())</code>
<code>list(T, T')</code>	<code>empty_list()</code> or <code>non_empty_list(T, T')</code>
<code>list(T)</code>	<code>empty_list()</code> or <code>non_empty_list(T, empty_list())</code>
<code>list()</code>	<code>empty_list()</code> or <code>non_empty_list(term(), empty_list())</code>

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
<code>{...}</code>	<code>tuple()</code>
<code>%{...}</code>	<code>open_map()</code>
<code>k : T</code>	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpecs to Elixir Types

$\llbracket \cdot \rrbracket : \text{TypeSpecs} \rightarrow \text{Elixir type}$

Figure 3: Semantic of Translation

3.1 Translation of TypeSpecs Specification into Elixir Annotations

$$\begin{aligned}
\llbracket @spec\ E \rrbracket &= \$ \llbracket E \rrbracket \\
\llbracket @spec\ \bar{E}, @spec\ E' \rrbracket &= \$ \llbracket \bar{E}, E' \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \text{ when } \bar{\alpha} : \bar{T}_3 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases} \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \text{ when } \bar{\alpha}' : \bar{T}'_3 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' = \emptyset \\ S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket [\alpha' := \beta'] \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\beta}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 4: Translation into Annotations

3.2 Translation of TypeSpecs into Elixir Types

$$\begin{aligned}
\llbracket \alpha \rrbracket &= \alpha \\
\llbracket T_1 \mid T_2 \rrbracket &= \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket \\
\llbracket \text{any}() \rrbracket &= \text{term}() \\
\llbracket \text{none}() \rrbracket &= \text{none}() \\
\llbracket \text{atom}() \rrbracket &= \text{atom}() \\
\llbracket :k \rrbracket &= :k \\
\llbracket \text{pid}() \rrbracket &= \text{pid}() \\
\llbracket \text{port}() \rrbracket &= \text{port}() \\
\llbracket \text{reference}() \rrbracket &= \text{reference}() \\
\llbracket \text{float}() \rrbracket &= \text{float}() \\
\llbracket \text{integer}() \rrbracket &= \text{integer}() \\
\llbracket \text{neg_integer} \rrbracket &= \text{infty}..-1 \\
\llbracket \text{non_neg_integer} \rrbracket &= 0..\text{infty} \\
\llbracket \text{pos_integer} \rrbracket &= 1..\text{infty} \\
\llbracket \text{tuple}() \rrbracket &= \text{tuple}() \\
\llbracket \{\bar{T}\} \rrbracket &= \{\llbracket \bar{T} \rrbracket\} \\
\llbracket \% \{F_1, \dots, F_n\} \rrbracket &= \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket) \\
\llbracket [] \rrbracket &= \text{empty_list}() \\
\llbracket \text{nonempty_maybe_improper_list}(T_1, T_2) \rrbracket &= \text{non_empty_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket) \\
\llbracket \langle\!\langle \rangle\!\rangle \rrbracket &= \text{bitstring}() \\
\llbracket \langle\!_::n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle\!_::_ * n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle\!_::n, _::_ * n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket (\bar{T} \rightarrow T) \rrbracket &= (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket) \\
\llbracket (\dots \rightarrow T) \rrbracket &= \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{none}() \rightarrow \llbracket T \rrbracket \text{ or } \dots \text{ or } \underbrace{(\text{none}(), \dots, \text{none}())}_{255 \text{ times}} \rightarrow \llbracket T \rrbracket & \text{otherwise} \end{cases} \\
\llbracket n \rrbracket &= n..n \\
\llbracket n..n' \rrbracket &= n..n'
\end{aligned}$$

Figure 5: Translation in Types

3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or $\text{if_set}(T)$. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by Empty the lists that maps all the non-singleton key types to $\text{not_set}()$.

First we define the translation of single field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 6:

$$\llbracket \text{required}(\mathsf{T}) \Rightarrow \mathsf{T}' \rrbracket = \begin{cases} \llbracket :k \Rightarrow \llbracket \mathsf{T}' \rrbracket \rrbracket & \text{if } \llbracket \mathsf{T} \rrbracket \approx :k \\ \llbracket \bigvee_{i \in I} \mathsf{K}_i \Rightarrow \text{if_set}(\llbracket \mathsf{T}' \rrbracket) \rrbracket & \text{if } \llbracket \mathsf{T} \rrbracket \approx \bigvee_{i \in I} \mathsf{K}_i \text{ and } \llbracket \mathsf{T} \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\llbracket \text{optional}(\mathsf{T}) \Rightarrow \mathsf{T}' \rrbracket = \begin{cases} \llbracket \bigvee_{i \in I} \mathsf{K}_i \Rightarrow \text{if_set}(\llbracket \mathsf{T}' \rrbracket) \rrbracket & \text{if } \llbracket \mathsf{T} \rrbracket \approx \bigvee_{i \in I} \mathsf{K}_i \\ \text{undefined} & \text{otherwise} \end{cases}$$

where all K_i 's in I are pairwise distinct.

Figure 6: Translation in Field Types

After obtaining types in Elixir Types we can define merge function. Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $\mathsf{K} \Rightarrow \tau$ of L_2 . Formally:

$$\begin{aligned} L_1 \oplus [] &= L_1 \\ L_1 \oplus [\mathsf{K} \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } \mathsf{K} \in L_1 \\ L_1 \oplus [\mathsf{K} \Rightarrow \tau, L_2] &= [\mathsf{K} \Rightarrow \tau, L_1] \oplus L_2 && \text{if } \mathsf{K} \notin L_1 \end{aligned}$$

Figure 7: Merge in Field Types

Notice that merging naturally reverses the order of field types, that is because field types in TypeSpecs are left-associative whereas those in Elixir Types are right-associative.

We then finally define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{ \mathsf{F}_1, \dots, \mathsf{F}_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket \mathsf{F}_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible key types), then we have $\llbracket \% \{ \mathsf{F}_1, \dots, \mathsf{F}_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket \mathsf{F}_i \rrbracket \oplus \text{Empty})$.

In spite of the whole translation process we have completed, there are three missing parts to be complemented: the merge of undefined translation; that of atom singleton types; and that of union types. In particular, these are the cases when two elements of merge function are not a disjunction. Accordingly, the proceeding section further extends the merge function to completion.

4 Total Approximating Translation of TypeSpecs into Elixir Types

The translation defined so far is partial since some TypeSpecs types do not have their exact corresponding types in Elixir Types. This is not a weakness of the Elixir Types system, but rather a consequence of the design choices made for Elixir Types, to keep them as simple as possible while still being useful. However, it is possible to define a total translation from types in TypeSpecs to ones in Elixir Types by approximating the original types in TypeSpecs. Total translation is useful in the context of migration, when we need to ensure that every type in TypeSpecs is represented in Elixir Types even approximately.

More precisely, the previous translations are *undefined* in the cases of translating field types; when $\llbracket \mathsf{T} \rrbracket$ in $\llbracket \text{required}(\mathsf{T}) \Rightarrow \mathsf{T}' \rrbracket$ or $\llbracket \text{optional}(\mathsf{T}) \Rightarrow \mathsf{T}' \rrbracket$ is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every type in TypeSpecs. The extension consists in defining approximations for the cases where the previous translation is undefined.

4.1 Approximation of Field Types

The merging case with conjunctive left-hand sides does not only perform merge function but also conduct approximation of types in TypeSpecs that are smaller than key types in Elixir Types. That is because while merge must take original intention of types in TypeSpecs into account, if we compare left-hand side types after blunt individual type approximation, in particular over-approximation, users' original aim could be deviated.

By taking that into account, we can safely write the promotion rule from TypeSpecs types that are *undefined* to key types in Elixir Types and the merge function with approximation.

To begin with, Let us define promotion function, $\text{promote}(\mathsf{F})$, which approximates $\llbracket \mathsf{T} \rrbracket$ to $\mathsf{K}^{\llbracket \mathsf{T} \rrbracket}$, i.e., the left-hand side type in field types to its lower bound supertype in key types. In other words, $\mathsf{K}^{\llbracket \mathsf{T} \rrbracket}$ is the key type promoted

from $\llbracket T \rrbracket$ which carries the information of its original type so that it can be compared not with the promoted type alone but with the intersection of the promoted type and the original type. This function is important for the merge function when applied to approximated field types in terms of comparison. Consider merging `1..2 => integer()` and `3..4 => float()`. If we promote `1..2` and `2..4` without the original information, the key type comparison is merely of `integer()` with `integer()`, resulting in completely overwriting the one that comes after with another. Whereas, with our promotion rule, the merge function can use the preserved information for the comparison keeping both right-hand side types in the given case; since the left-hand side original types are disjunctive. Hence, to rewrite the translation of field types after applying the promotion function:

$$\text{promote}(\llbracket \text{required}(T) \Rightarrow T' \rrbracket) = \begin{cases} \llbracket :k \Rightarrow \llbracket T' \rrbracket \rrbracket & \text{if } \llbracket T \rrbracket \approx :k \\ \llbracket \bigvee_{\{i \in I; \llbracket T_i \rrbracket \cap K_i \neq \emptyset\}} K_i^{\llbracket T_i \rrbracket} \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \rrbracket & \text{if } \llbracket T \rrbracket \not\approx :k \end{cases}$$

$$\text{promote}(\llbracket \text{optional}(T) \Rightarrow T' \rrbracket) = \llbracket \bigvee_{\{i \in I; \llbracket T_i \rrbracket \cap K_i \neq \emptyset\}} K_i^{\llbracket T_i \rrbracket} \Rightarrow \text{if_set}(\llbracket T' \rrbracket) \rrbracket$$

Figure 8: Approximated Translation in Field Types

The merge function extends its definition further with the extra information from approximated translation. For convenience, we substitute $\llbracket T \rrbracket$ with s throughout our definition.

Trivially, merging a list of fields to an empty list is the same as the original function producing the list.

In the second and third definitions, we first look for the same key type of the field type in the list. If one exists, we get an intersection of the key type and the original type retrieved from the list, and get another intersection of the field type. Eventually, compare the derived types and find the subtype relations. The second definition is when the the intersection from field type is a subtype of the one from the list, in which case the field type is simply eradicated. On the other hand if we find the comparison not in a subtype relation, we append the field type to the list with two union operations. That are unification of original types and that of right-hand side types. Lastly, the field type from the list that the found key type belonged is eradicated.

For the final definition of merge function, all we need is to find in the list non-existence of the exactly same left-hand side type from the field type. Then we can simply add the new field type to the list.

$$\begin{aligned} \llbracket L_1 \rrbracket \oplus \llbracket \rrbracket &= \llbracket L_1 \rrbracket \\ L_1 \oplus \llbracket K^s \Rightarrow \tau, L_2 \rrbracket &= L_1 \oplus L_2 && \text{if } K^{s_1} \in L_1 \text{ and } K \cap s \leq K \cap s_1 \\ L_1 \oplus \llbracket K^s \Rightarrow \tau, L_2 \rrbracket &= \llbracket K^{s|s_1} \Rightarrow \tau \text{ or } \tau_1, L_1 \setminus K^{s_1} \Rightarrow \tau_1 \rrbracket \oplus L_2 && \text{if } K^{s_1} \in L_1 \text{ and } K \cap s \not\leq K \cap s_1 \\ L_1 \oplus \llbracket K^s \Rightarrow \tau, L_2 \rrbracket &= \llbracket K^s \Rightarrow \tau, L_1 \rrbracket \oplus L_2 && \text{if } \nexists s_1 \text{ } K^{s_1} \in L_1 \end{aligned}$$

Figure 9: Merge in Approximated Field Types

At last, we revise the field type translation to $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \text{promote}(\llbracket F_i \rrbracket))$, and by merging Empty to that end of merges, we obtain a saturated map, $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \text{promote}(\llbracket F_i \rrbracket) \oplus \text{Empty})$.